Business Case: TargetSQL

SANDEEP KUMAR APKANI

31-07-2024

Context:

Target is a globally renowned brand and a prominent retailer in the United States. Target makes itself a preferred shopping destination by offering outstanding value, inspiration, innovation and an exceptional guest experience that no other retailer can deliver.

This particular business case focuses on the operations of Target in Brazil and provides insightful information about 100,000 orders placed between 2016 and 2018. The dataset offers a comprehensive view of various dimensions including the order status, price, payment and freight performance, customer location, product attributes, and customer reviews.

By analyzing this extensive dataset, it becomes possible to gain valuable insights into Target's operations in Brazil. The information can shed light on various aspects of the business, such as order processing, pricing strategies, payment and shipping efficiency, customer demographics, product characteristics, and customer satisfaction levels.

The data is available in 8 csv files:

1. customers.csv
2. sellers.csv
3. order_items.csv
4. geolocation.csv
5. payments.csv
6. reviews.csv
7. orders.csv
8. products.csv

The column description for these csv files is given below.

The **customers.csv** contain following features:

Features	**Description**
customer_id	ID of the consumer who made the purchase
customer_unique_id	Unique ID of the consumer
customer_zip_code_prefix	Zip Code of consumer's location
customer_city	Name of the City from where order is made
customer_state	State Code from where order is made (Eg. são paulo - SP)

The **sellers.csv** contains following features:

Features	**Description**
seller_id	Unique ID of the seller registered
seller_zip_code_prefix	Zip Code of the seller's location
seller_city	Name of the City of the seller
seller_state	State Code (Eg. são paulo - SP)

The **order_items.csv** contain following features:

Features	**Description**
order_id	A Unique ID of order made by the consumers
order_item_id	A Unique ID given to each item ordered in the order
product_id	A Unique ID given to each product available on the site
seller_id	Unique ID of the seller registered in Target
shipping_limit_date	The date before which the ordered product must be shipped
price	Actual price of the products ordered

freight_value	Price rate at which a product is delivered from one point to another

The **geolocations.csv** contain following features:

Features	**Description**
geolocation_zip_code_prefix	First 5 digits of Zip Code
geolocation_lat	Latitude
geolocation_lng	Longitude
geolocation_city	City
geolocation_state	State

The **payments.csv** contain following features:

Features	**Description**
order_id	A Unique ID of order made by the consumers
payment_sequential	Sequences of the payments made in case of EMI
payment_type	Mode of payment used (Eg. Credit Card)
payment_installments	Number of installments in case of EMI purchase
payment_value	Total amount paid for the purchase order

The **orders.csv** contain following features:

Features	**Description**
order_id	A Unique ID of order made by the consumers
customer_id	ID of the consumer who made the purchase
order_status	Status of the order made i.e. delivered, shipped, etc.
order_purchase_timestamp	Timestamp of the purchase
order_delivered_carrier_date	Delivery date at which carrier made the delivery
order_delivered_customer_date	Date at which customer got the product
order_estimated_delivery_date	Estimated delivery date of the products

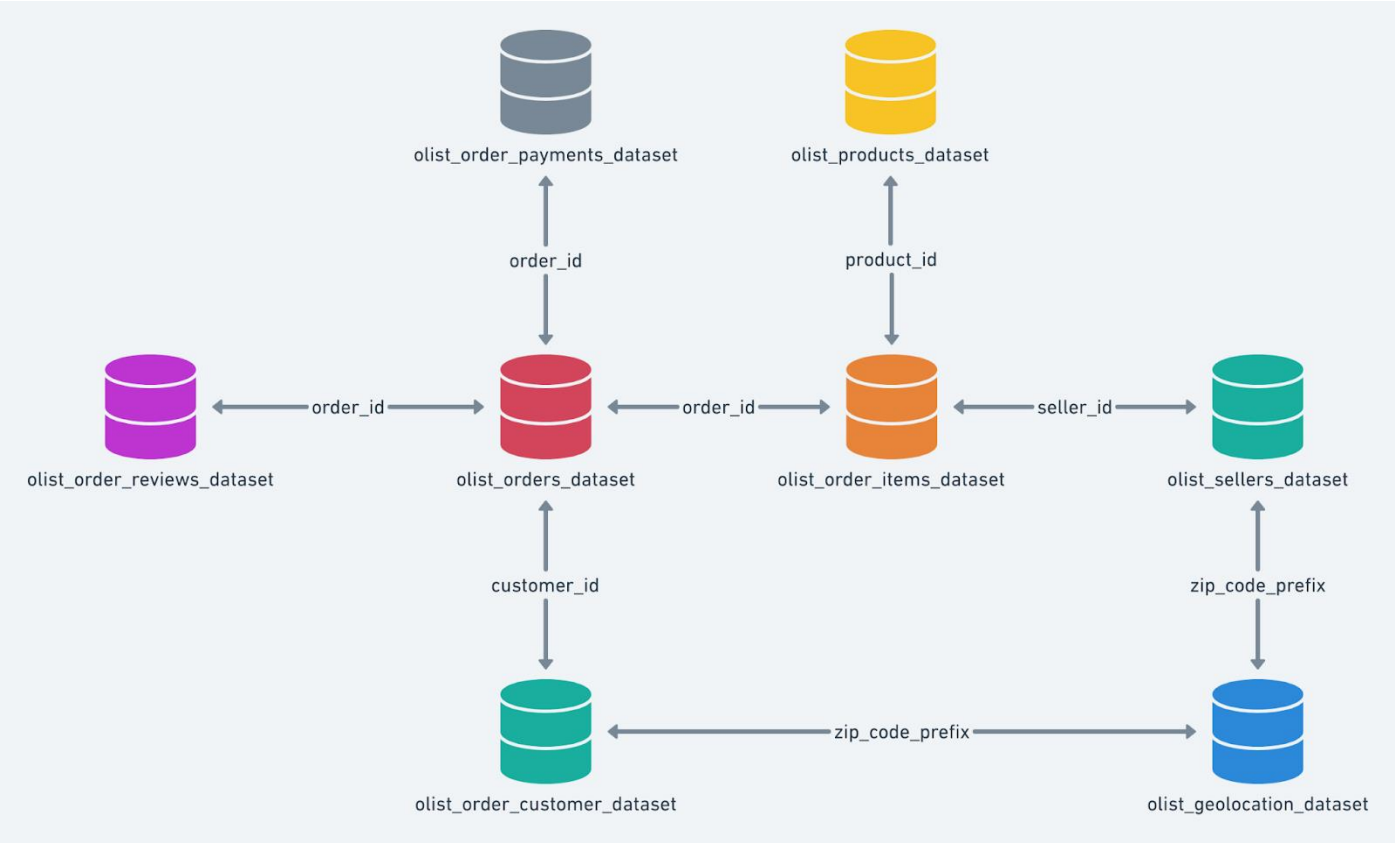
The **reviews.csv** contain following features:

Features	**Description**
review_id	ID of the review given on the product ordered by the order id
order_id	A Unique ID of order made by the consumers
review_score	Review score given by the customer for each order on a scale of 1-5
review_comment_title	Title of the review
review_comment_message	Review comments posted by the consumer for each order
review_creation_date	Timestamp of the review when it is created
review_answer_timestamp	Timestamp of the review answered

The **products.csv** contain following features:

Features	**Description**
product_id	A Unique identifier for the proposed project.
product_category_name	Name of the product category
product_name_lenght	Length of the string which specifies the name given to the products ordered
product_description_lenght	Length of the description written for each product ordered on the site
product_photos_qty	Number of photos of each product ordered available on the shopping portal
product_weight_g	Weight of the products ordered in grams
product_length_cm	Length of the products ordered in centimeters
product_height_cm	Height of the products ordered in centimeters
product_width_cm	Width of the product ordered in centimeters

Dataset schema:



Problem Statement:

Assuming you are a data analyst/ scientist at Target, you have been assigned the task of analyzing the given dataset to extract valuable insights and provide actionable recommendations.

1.1. Data type of all columns in the "customers" table

Table. 01

Filter Enter property name or value

Field name	Type	Mode	Key	Collation
customer_id	STRING	NULLABLE	-	-
customer_unique_id	STRING	NULLABLE	-	-
customer_zip_code_prefix	INTEGER	NULLABLE	-	-
customer_city	STRING	NULLABLE	-	-
customer_state	STRING	NULLABLE	-	-

Inference: From the above, we able know the data type of the customer table.

1.2 Get the time range between which the orders were placed.

```
SELECT
      MIN(order_purchase_timestamp) AS min_order_date,
     MAX(order_purchase_timestamp) AS max_order_date
FROM
 target.orders
```

Table. 02

JOB INFORMATION | RESULTS | CHART | JSON

Row	min_order_date	max_order_date
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC

Inference: Based on the data provided the range is between 2016-09-04 and 2018-10-17. This dataset has data containing about 3 years.

1.3 Count the Cities & States of customers who ordered during the given period

```
SELECT
  COUNT(DISTINCT customer_city) AS num_of_cities,
  COUNT(DISTINCT customer_state) AS num_of_states
FROM   target.customers
```

Table. 03

JOB INFORMATION | RESULTS | CHART

Row	num_of_cities	num_of_states
1	4119	27

Inference: From the provided customers data. The people from the 4,119 cities and 27 sates have purchase history from the Brazil country.

2.1 Is there a growing trend in the no. of orders placed over the past years?

```
SELECT year, COUNT(year) as num_of_orders
FROM
(SELECT order_purchase_timestamp,extract(year from
order_purchase_timestamp) as year
FROM target.orders) as tab1
GROUP BY tab1.year
ORDER BY year ASC;
```

Table. 04

Row	year	num_of_orders
1	2016	329
2	2017	45101
3	2018	54011

Inference: from the provided dataset the sales trend based on the orders per year. There is increasing trend in the order places year by year in the ascending order. During the initial year the sales are 329 and sharp increase in year 2017 from 329 to 45,101 and in year 2018 there is slight increase compared with last year from 45,101 to 54,011

2.2 Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

For year -2016

```
with cte1 as (SELECT order_id,order_purchase_timestamp, EXTRACT(year
FROM _purchase_timestamp) as order_year,
EXTRACT(month from order_purchase_timestamp) as order_month
FROM target.orders),

cte2 as (SELECT order_year, order_month,
```

count(order_id) as num_orders_month

FROM cte1

WHERE order_year = 2016

GROUP BY order_year, order_month)

SELECT * from cte2

ORDER BY order_month

Table. 05

Row	order_year	order_month	num_orders_month
1	2016	9	4
2	2016	10	324
3	2016	12	1

FOR YEAR = 2017

with cte1 as (SELECT order_id,order_purchase_timestamp, EXTRACT(year FROM _purchase_timestamp) as order_year,

EXTRACT(month from order_purchase_timestamp) as order_month

FROM target.orders),

cte2 as (SELECT order_year, order_month,

count(order_id) as num_orders_month

FROM cte1

WHERE order_year = 2017

GROUP BY order_year, order_month)

SELECT * from cte2 ORDER BY order_month

Table. 06

Row	order_year	order_month	num_orders_month
1	2017	1	800
2	2017	2	1780
3	2017	3	2682
4	2017	4	2404
5	2017	5	3700
6	2017	6	3245
7	2017	7	4026
8	2017	8	4331
9	2017	9	4285
10	2017	10	4631
11	2017	11	7544
12	2017	12	5673

FOR YEAR = 2018

```
with cte1 as (SELECT order_id,order_purchase_timestamp,
EXTRACT(year FROM _purchase_timestamp) as order_year,
EXTRACT(month from order_purchase_timestamp) as order_month
FROM target.orders),

cte2 as (SELECT order_year, order_month,
count(order_id) as num_orders_month
FROM cte1
WHERE order_year = 2018
GROUP BY order_year, order_month)
SELECT * from cte2
ORDER BY order_month
```

Table. 07

Row	order_year	order_month	num_orders_month
1	2018	1	7269
2	2018	2	6728
3	2018	3	7211
4	2018	4	6939
5	2018	5	6873
6	2018	6	6167
7	2018	7	6292
8	2018	8	6512
9	2018	9	16
10	2018	10	4

Inference: comparing the three years of data month-on-month data. There is no specific seasonality trend form the data. The data is order by month wise in the ascending order to compare with respective years.

2.3 During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)

- 0-6 hrs : Dawn
- 7-12 hrs : Mornings
- 13-18 hrs : Afternoon
- 19-23 hrs : Night

```
WITH CTE1 as (SELECT order_id, order_purchase_timestamp,
 EXTRACT(hour FROM order_purchase_timestamp) as order_hour
 FROM target.orders),

 CTE2 as (SELECT order_id, order_hour,
        CASE
        WHEN order_hour BETWEEN 0 AND 6 THEN  "Dawn"
        WHEN order_hour BETWEEN 7 AND 12 THEN  "Morning"
        WHEN order_hour BETWEEN 13 AND 18 THEN  "After noon"
        ELSE "Night"
        END as order_time
       from CTE1)

SELECT distinct order_time, count(order_id) OVER (PARTITION BY order_time) as count_ordes_time
FROM cte2
ORDER BY order_time
```

Table. 08

Row	order_time	count_ordes_time
1	After noon	38135
2	Dawn	5242
3	Morning	27733
4	Night	28331

Inference: By analysing the data of three years of Brazilian customers. We are able to know that the people are mostly placing the orders during Morning hour.

3.1 Get the month-on-month no. of orders placed in each state.

```
select order_year, order_month, count(1) as month_sales
from (select order_purchase_timestamp,
            date(order_purchase_timestamp) as order_date,
           extract(year from order_purchase_timestamp) as order_year,
           extract(month from order_purchase_timestamp) as order_month
FROM target.orders) as tab1
GROUP BY order_year, order_month
OREDR BY order_year, order_month
```

Table. 09

Row	order_year	order_month	month_sales
1	2016	9	4
2	2016	10	324
3	2016	12	1
4	2017	1	800
5	2017	2	1780
6	2017	3	2682
7	2017	4	2404
8	2017	5	3700
9	2017	6	3245
10	2017	7	4026

Inference: From the above provided data. The query results in every year the number of sales done month-on-month. Are provided the data. The result is ordered by year and month wise respectively.

3.2 How are the customers distributed across all the states?

```
SELECT DISTINCT customer_state as state_name,
  COUNT(customer_id) OVER (PARTITION BY customer_state) AS
  num_of_customers
FROM   target.customers
```

Table. 10

Row	state_name	num_of_customers
1	RR	46
2	AP	68
3	MS	715
4	DF	2140
5	PA	975
6	SC	3637
7	RS	5466
8	MG	11635
9	AC	81
10	PB	536

Inference: From the given data, the number of the customers (num_of_customers) spread across each state (state_name) shown in the above table respectively.

4.1 Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).

You can use the "payment_value" column in the payments table

```
WITH ct1 AS     (SELECT p.order_id, p.payment_value,
                 o.order_status, o.order_purchase_timestamp,
                 extract(year from o.order_purchase_timestamp) as
purchase_year,
                 extract(month from o.order_purchase_timestamp) AS
purchase_month
        FROM target.payments p INNER JOIN target.orders o
            ON p.order_id  = o.order_id),
            --where o.order_status = "shipped"),

    cte2 AS  (select  purchase_year,
              sum(payment_value) as cost_of_order
              from ct1
```

```
where purchase_month BETWEEN 1 AND 8 AND
 purchase_year BETWEEN 2017 AND 2018
 group by purchase_year ),

cte3 AS      (select purchase_year,cost_of_order,
        lag(cost_of_order) OVER (order by purchase_year) as lead_,
            (cost_of_order -lag(cost_of_order) OVER (order by
purchase_year ) ) as diff_value
         from cte2)

SELECT purchase_year,cost_of_order,
      ROUND((cost_of_order/diff_value)*100,3) as
percentage_increase
      from cte3
ORDER BY purchase_year
```

Table. 11

Row	purchase_year	cost_of_order	percentage_increase
1	2017	3669022.120000...	null
2	2018	8694733.839999...	173.005

Inference: The table (table. 11) is showing the sales percentage increased between the two years and months from January to August.

4.2 Calculate the Total & Average value of order price for each state.

```
SELECT DISTINCT c.customer_state,
      ROUND(SUM(payment_value) OVER(PARTITION BY
customer_state),2) AS total_order_price,
      ROUND(AVG(payment_value) OVER (PARTITION BY
customer_state),2) AS total_avg_value
FROM target.customers c INNER JOIN  target.orders o
     ON c.customer_id = o.customer_id INNER JOIN target.payments p
     ON  o.order_id = p.order_id
```

ORDER BY customer_state

Table. 11

Row	customer_state	total_sales	total_avg
1	AC	19680.62	234.29
2	AL	96962.06	227.08
3	AM	27966.93	181.6
4	AP	16262.8	232.33
5	BA	616645.82	170.82
6	CE	279464.03	199.9
7	DF	355141.08	161.13
8	ES	325967.55	154.71
9	GO	350092.31	165.76
10	MA	152523.02	198.86

Inference: The above table is showing the total number of sales(total_sales) and average of total sales (total_avg) made for each respective state.

4.3 Calculate the Total & Average value of order freight for each state

```
SELECT DISTINCT customer_state,
    round(sum(oi.freight_value) over (partition by c.customer_state),2) as total_fright_value,
    round(avg(oi.freight_value) over (partition by c.customer_state),2) as avg_fright_value
    FROM target.customers AS c INNER JOIN target.orders AS o
    ON c.customer_id = o.customer_id
    INNER JOIN target.order_items as oi
    ON  o.order_id = oi.order_id
order by customer_state
```

Table. 12

Row	customer_state	total_fright_value	avg_fright_value
1	AC	3686.75	40.07
2	AL	15914.59	35.84
3	AM	5478.89	33.21
4	AP	2788.5	34.01
5	BA	100156.68	26.36
6	CE	48351.59	32.71
7	DF	50625.5	21.04
8	ES	49764.6	22.06
9	GO	53114.98	22.77
10	MA	31523.77	38.26

Inference: The above table is showing the total sum of freight value (total_freight_value) of the orders state wise and average freight value(avg_freight_value)

5.1 Find the no. of days taken to deliver each order from the order's purchase date as delivery time.
Also, calculate the difference (in days) between the estimated & actual delivery date of an order.
Do this in a single query.

You can calculate the delivery time and the difference between the estimated & actual delivery date using the given formula:

- **time_to_deliver** = order_delivered_customer_date - order_purchase_timestamp
- **diff_estimated_delivery** = order_delivered_customer_date - order_estimated_delivery_date

```
WITH    delivered as      (SELECT order_id, order_status,
        order_delivered_customer_date,
        order_purchase_timestamp,   order_estimated_delivery_date

        FROM target.orders

        where order_status = "delivered"),


       days_diff as (select  order_id, order_status,

       date_diff(order_delivered_customer_date,order_purchase_timestam
       p, day) as time_to_deliver,

       date_diff(order_delivered_customer_date,
       order_estimated_delivery_date, day) as
       diff_estimated_delivery   from delivered)
```

```
SELECT  * from days_diff
ORDER BY diff_estimated_delivery DESC
```

Table. 13

Row	order_id	order_status	time_to_deliver	diff_estimated_deliv
1	1b3190b2dfa9d789e1f14c05b...	delivered	208	188
2	ca07593549f1816d26a572e06...	delivered	209	181
3	47b40429ed8cce3aee9199792...	delivered	191	175
4	2fe324febf907e3ea3f2aa9650...	delivered	189	167
5	285ab9426d6982034523a855f...	delivered	194	166
6	440d0d17af552815d15a9e41a...	delivered	195	165
7	c27815f7e3dd0b926b5855262...	delivered	187	162
8	0f4519c5f1c541ddec9f21b3bd...	delivered	194	161
9	d24e8541128cea179a11a6517...	delivered	175	161
10	2d7561026d542c8dbd8f0daea...	delivered	188	159

Inference: The data is showing the places order wise where the order_status is shipped. The tame taken for each delivery and the difference in the estimated time of delivery.

5.2 Find out the top 5 states with the highest & lowest average freight value.

Highest average fright charge

```
SELECT DISTINCT customer_state,
        round(avg(oi.freight_value) over (partition by
        c.customer_state),3) as total_fright_value,
FROM target.customers AS c INNER JOIN target.orders AS o
        ON c.customer_id = o.customer_id
        INNER JOIN target.order_items as oi
        ON  o.order_id = oi.order_id
        ORDER BY total_fright_value DESC
        LIMIT 5
```

Table. 14

Row	customer_state	total_fright_value
1	RR	42.984
2	PB	42.724
3	RO	41.07
4	AC	40.073
5	PI	39.148

Lowest average fright charge:

```
SELECT DISTINCT customer_state,
        round(avg(oi.freight_value) over (partition by
        c.customer_state),3) as total_fright_value,
FROM target.customers AS c INNER JOIN target.orders AS o
        ON c.customer_id = o.customer_id
        INNER JOIN target.order_items as oi
        ON o.order_id = oi.order_id
        ORDER BY total_fright_value ASC
        LIMIT 5
```

Table. 15

Row	customer_state	total_fright_value
1	SP	15.147
2	PR	20.532
3	MG	20.63
4	RJ	20.961
5	DF	21.041

Inference: the above two tables are showing top and lowest five states having average freight value.

5.3 Find out the top 5 states with the highest & lowest average delivery time

Highest average deliver time:

```
SELECT DISTINCT customer_state,
        AVG(date_diff(order_delivered_customer_date,
        order_purchase_timestamp, day)) over (partition by c.customer_state)
        as avg_delivery_time
FROM target.customers AS c INNER JOIN target.orders AS o
            ON c.customer_id = o.customer_id
        ORDER BY avg_delivery_time desc
        LIMIT 5
```

Table. 16

Row	customer_state	avg_delivery_time
1	RR	28.97560975609...
2	AP	26.73134328358...
3	AM	25.98620689655...
4	AL	24.04030226700...
5	PA	23.31606765327...

Lowest average delivery time:

```
SELECT DISTINCT customer_state,
        AVG(date_diff(order_delivered_customer_date,
        order_purchase_timestamp, day)) over (partition by c.customer_state)
        as avg_delivery_time
FROM target.customers AS c INNER JOIN target.orders AS o
            ON c.customer_id = o.customer_id
        ORDER BY avg_delivery_time asc
        LIMIT 5
```

Table. 17

Row	customer_state	avg_delivery_time
1	SP	8.298061489072...
2	PR	11.52671135486...
3	MG	11.54381329810...
4	DF	12.50913461538...
5	SC	14.47956019171...

Inference: The two tables are showing the top and lowest five states having average delivery time of the products.

5.4 Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.

You can use the difference between the averages of actual & estimated delivery date to figure out how fast the delivery was for each state.

```
SELECT DISTINCT customer_state,
    AVG(date_diff(order_estimated_delivery_date,
    order_delivered_customer_date, day)) over (partition by
    c.customer_state) as top_fastest_delivery
FROM target.customers AS c INNER JOIN target.orders AS o
    ON c.customer_id = o.customer_id
    ORDER BY top_fastest_delivery ASC
    LIMIT 5
```

Table. 18

Row	customer_state	top_fastest_delivery
1	AL	7.947103274559...
2	MA	8.768479776847...
3	SE	9.173134328358...
4	ES	9.618546365914...
5	BA	9.934889434889...

Inference: The above table is showing the top 5 state names data where the sates received the ordered products earlier compared to with the estimated delivery.

6 Analysis based on the payments:

6.1 Find the month on month no. of orders placed using different payment types.

```
WITH cte1 AS (select payment_type, extract(year
from  order_purchase_timestamp) as year, extract(month from
order_purchase_timestamp) as month
        FROM target.orders o inner join target.payments p
                ON o.order_id = p.order_id),

        cte2 AS (select year, month,payment_type, count(1) as num_orders
                from cte1 group by payment_type, year, month)
                SELECT  * from cte2
                ORDER BY year,month, payment_type
```

Table. 19

Row	year	month	payment_type	num_orders
1	2016	9	credit_card	3
2	2016	10	UPI	63
3	2016	10	credit_card	254
4	2016	10	debit_card	2
5	2016	10	voucher	23
6	2016	12	credit_card	1
7	2017	1	UPI	197
8	2017	1	credit_card	583
9	2017	1	debit_card	9
10	2017	1	voucher	61

Inference: The above table is showing number of orders placed month wise using different modes payments types like credit card, debit card, UPI etc.

6.2 Find the no. of orders placed on the basis of the payment installments that have been paid.

```
SELECT DISTINCT payment_installments,
COUNT(order_id) over (partition by payment_installments) as number_of_orders
FROM target.payments p
ORDER BY payment_installments ASC
```

Table. 20

Row	payment_installment	number_of_orders
1	0	2
2	1	52546
3	2	12413
4	3	10461
5	4	7098
6	5	5239
7	6	3920
8	7	1626
9	8	4268
10	9	644

Inference: This above table is showing the total number of customers who opted to pay bills in every month instalment (EMI). This show range of EMIs opted by customers.